agentic-nlq-eval

Student Technical Report

A colorful, easy-English guide to the architecture,
systems, data flow, experiments, and scoring.

What you will learn

1. What natural-language querying means
2. How S1, S2, and S3 differ
3. How data, engines, and the LLM fit together
4. How experiments stay fair and reproducible
5. How answers are graded

Based on project version 0.1.0 | Research harness (not a product UI)

Contents

* 1. Welcome: what this project is (in plain English)
* 2. Big-picture diagram
* 3. System architecture layers
* 4. Code map (folders)
* 5. Data flow and business views
* 6. The three answering systems (S1, S2, S3)
* 7. S3 LangGraph flowchart (detailed)
* 8. LLM client, prompts, and budget safety
* 9. SQL validation (the safety gate)
* 10. Experiment workflow
* 11. Evaluation: accuracy, faithfulness, latency
* 12. Configuration and secrets
* 13. Student learning path
* 14. Glossary

1. Welcome: what this project is

Normal people ask questions in English. Databases understand SQL (a special query language). A natural-language query (NLQ) system is a bridge: it reads your English question, writes SQL, runs it, and explains the answer.

agentic-nlq-eval is a research lab for that bridge. It is not a shiny website for customers. It is a fair race track where three designs compete on the same questions and the same data, so we can measure which design is better.

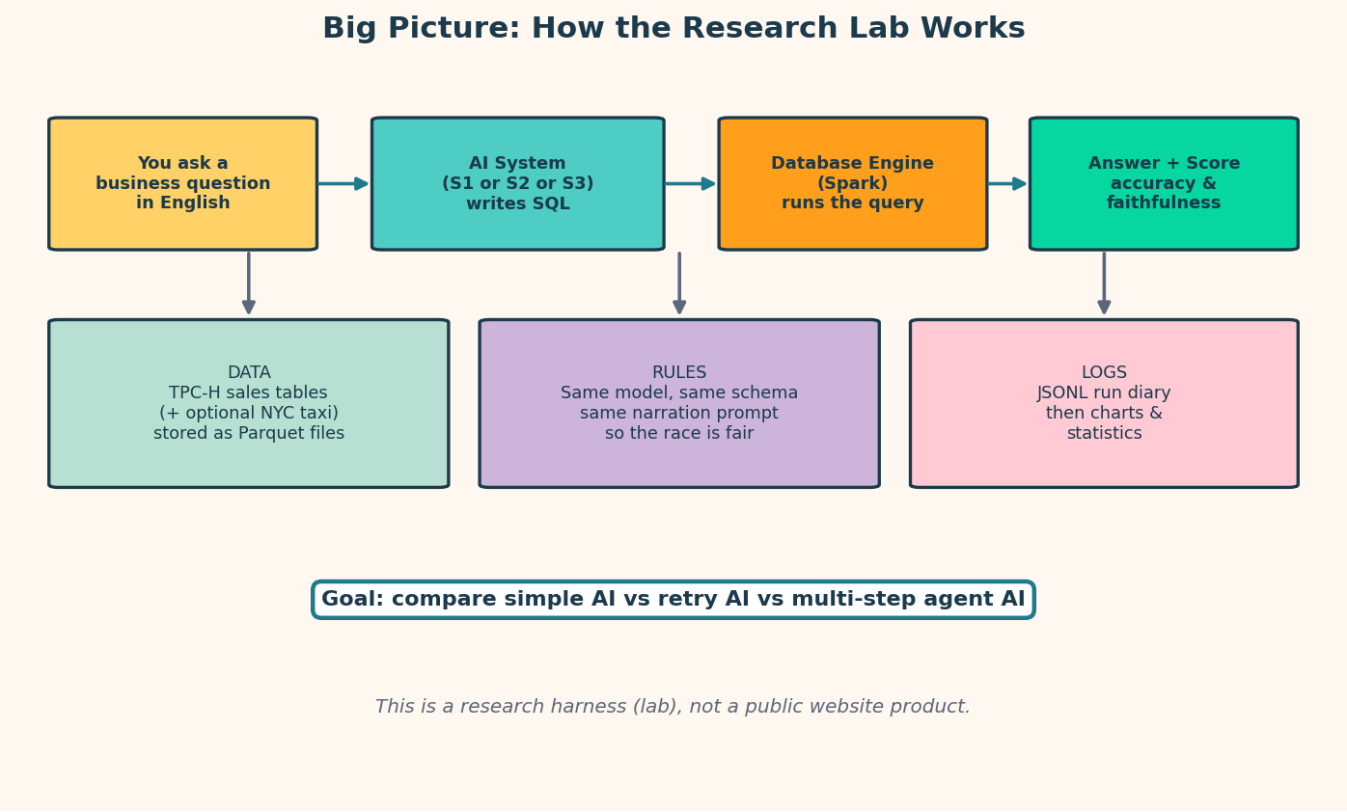
The three racers (short version)

* S1: one try - write SQL, run it, explain. Fast and simple.
* S2: like S1, but if SQL fails, show the error and try again (up to 3 retries).
* S3: a multi-step agent (LangGraph) that plans, validates, executes, explains, and checks claims.

Student tip: If this feels big, treat it like cooking: S1 is one recipe step, S2 is tasting and fixing, S3 is a full kitchen checklist with a quality inspector at the end.

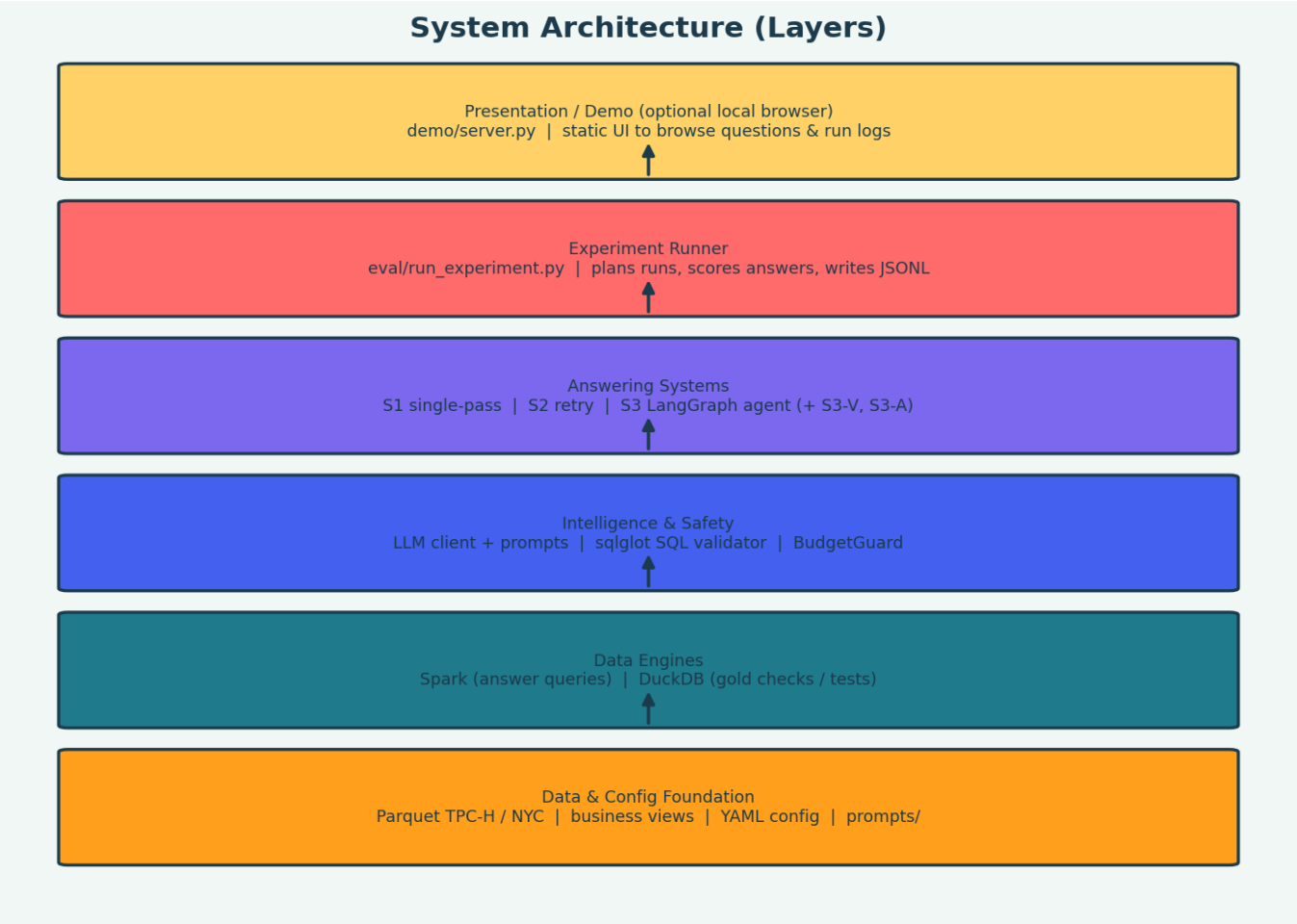
2. Big-picture diagram

Read this diagram left to right. A question goes into an AI system. The system writes SQL. Spark runs SQL on Parquet data. We score the answer. Underneath, shared data, fair rules, and logs keep the race honest.



3. System architecture layers

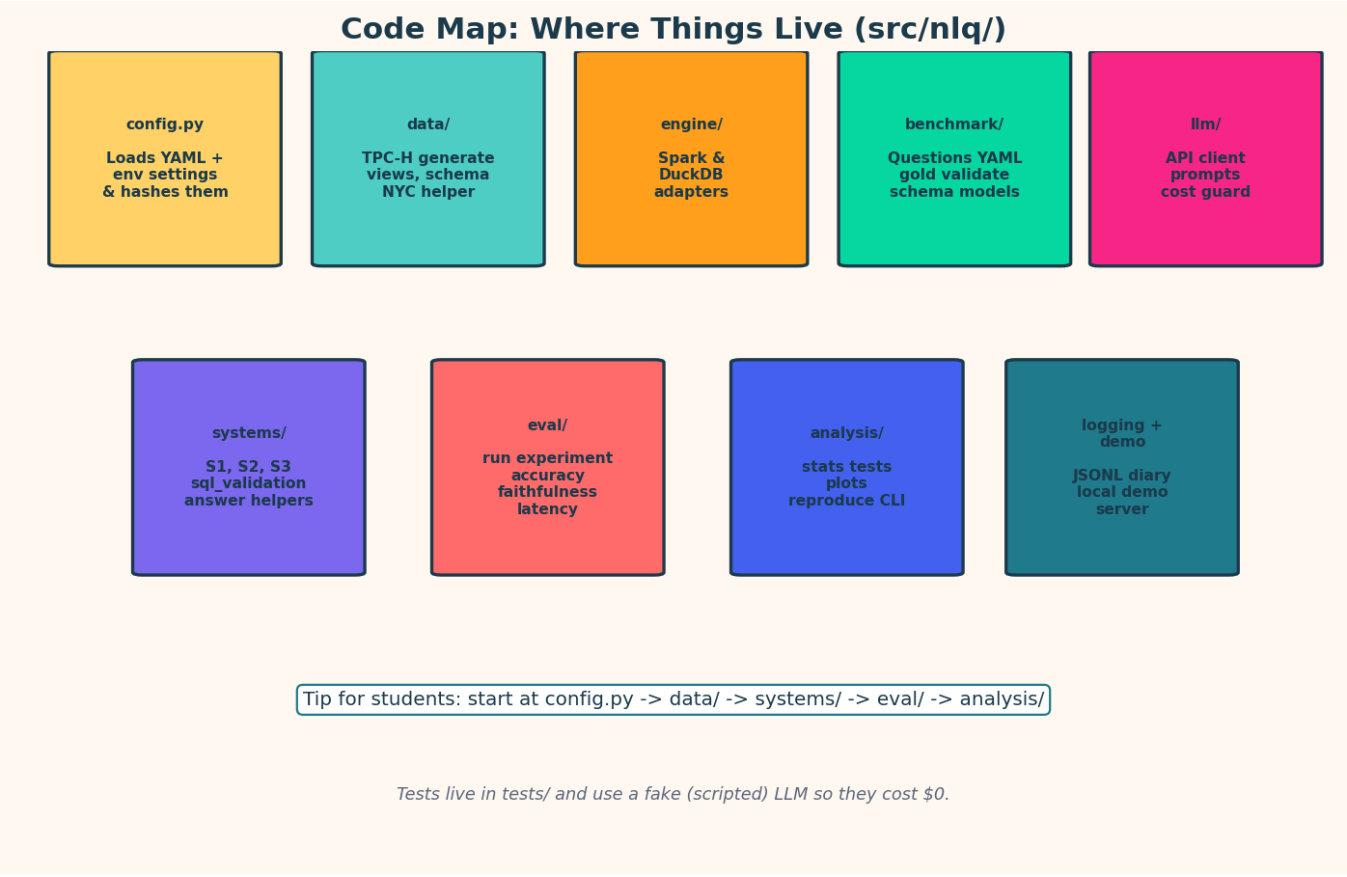
Think of the software as a stack of layers. The bottom holds data and config. The middle talks to databases and the AI model. Near the top, S1/S2/S3 answer questions. The runner runs experiments. An optional demo server only helps you look at results.



Important rule for fairness: all systems share one model, one schema description, and one narration prompt style. We change the control flow, not the 'brain brand'.

4. Code map (folders)

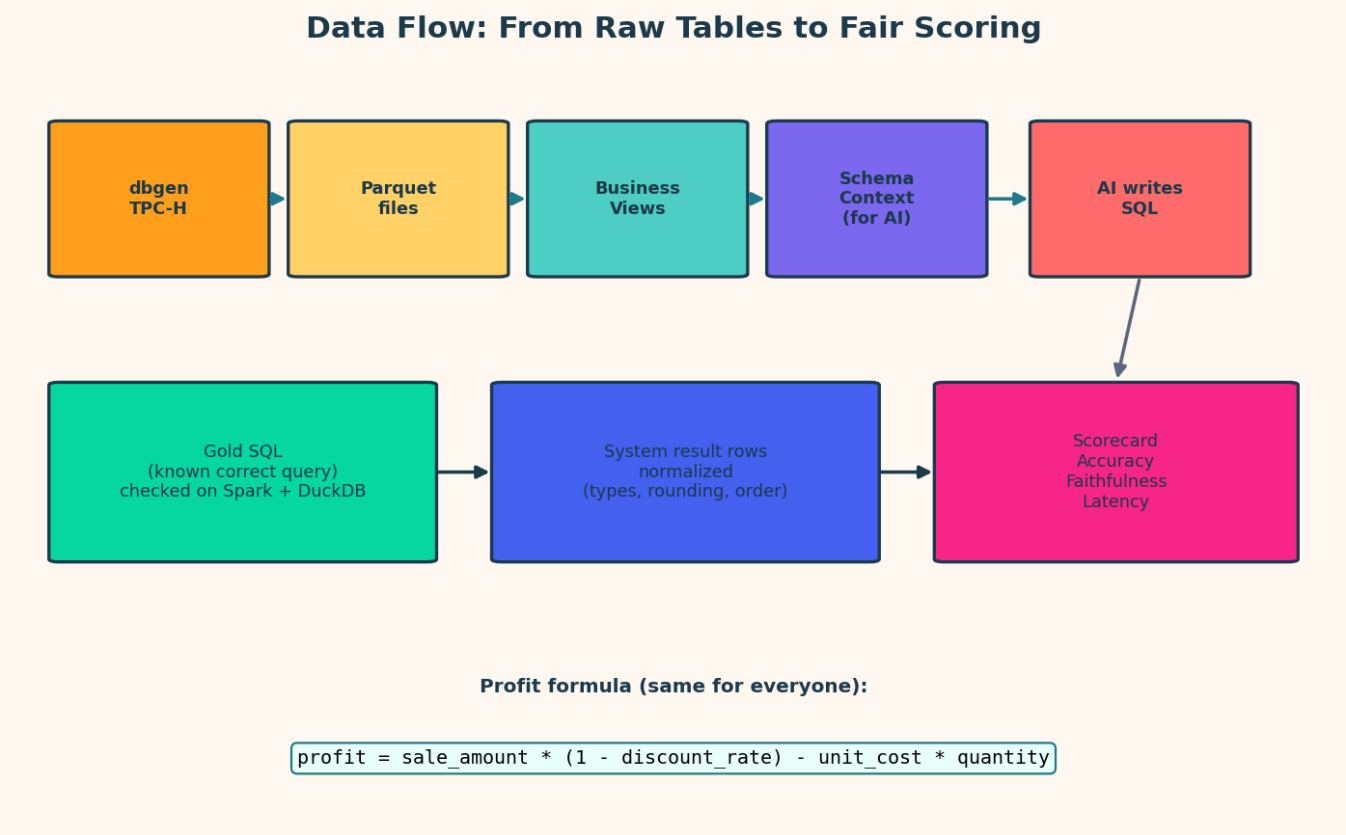
When you open the repo, almost everything useful is under src/nlq/. This map shows which folder owns which job.



5. Data flow and business views

TPC-H is a standard fake sales/supply-chain dataset. We can generate it at different sizes (scale factors). Small sizes are for quick tests. Bigger sizes stress performance.

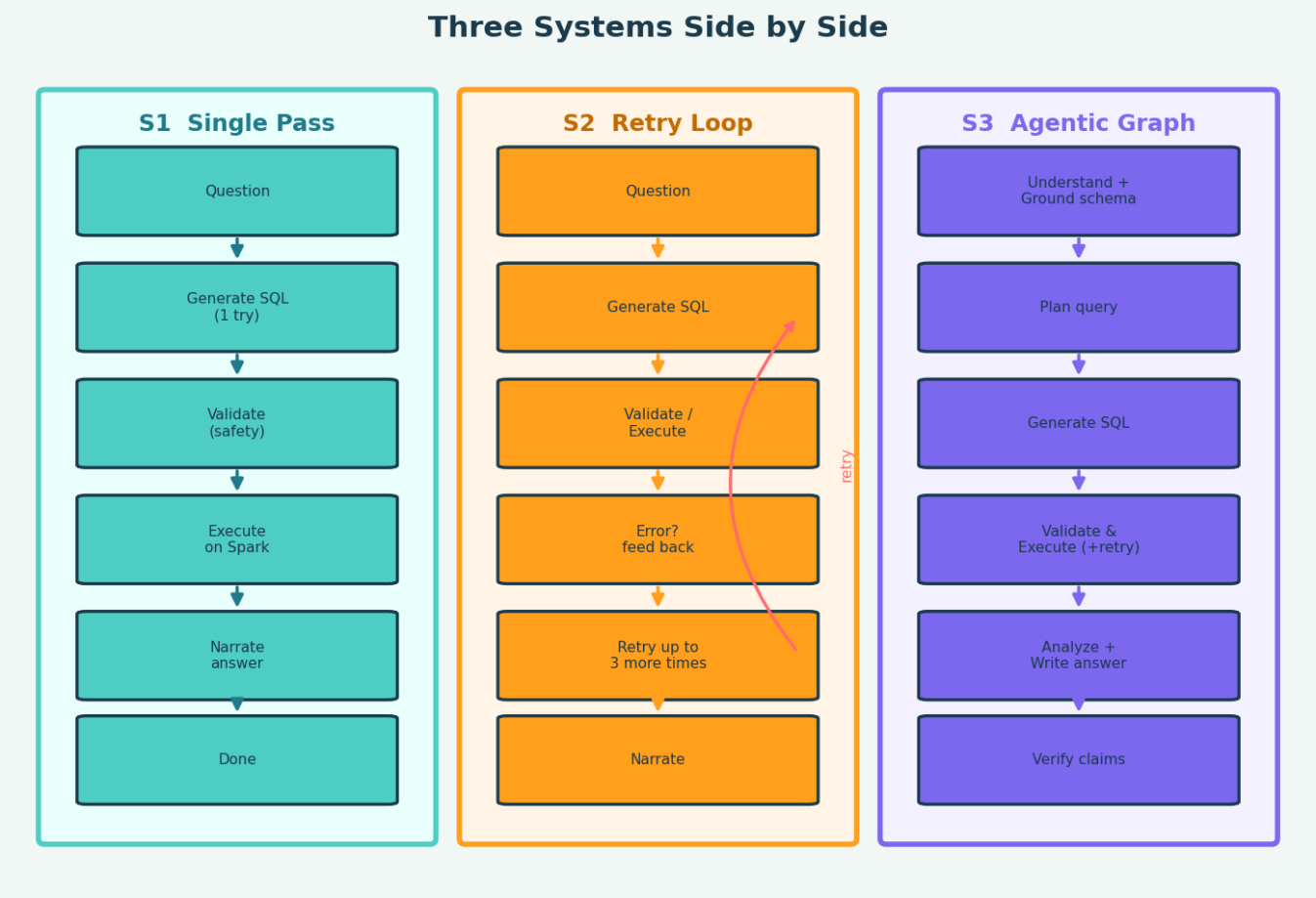
The AI never sees ugly raw TPC-H names. The engine creates friendly business views such as customers, suppliers, sales_orders, and order_lines. That matches how a business analyst thinks.



* Parquet = columnar file format, good for analytics.
* Spark runs the answering SQL in experiments.
* DuckDB helps validate gold answers and supports tests.
* Gold answer = known-correct reference used for scoring.

6. The three answering systems

Same question, same data, three strategies. Study the middle column carefully: S2's power is the feedback loop. S3's power is many specialized steps.

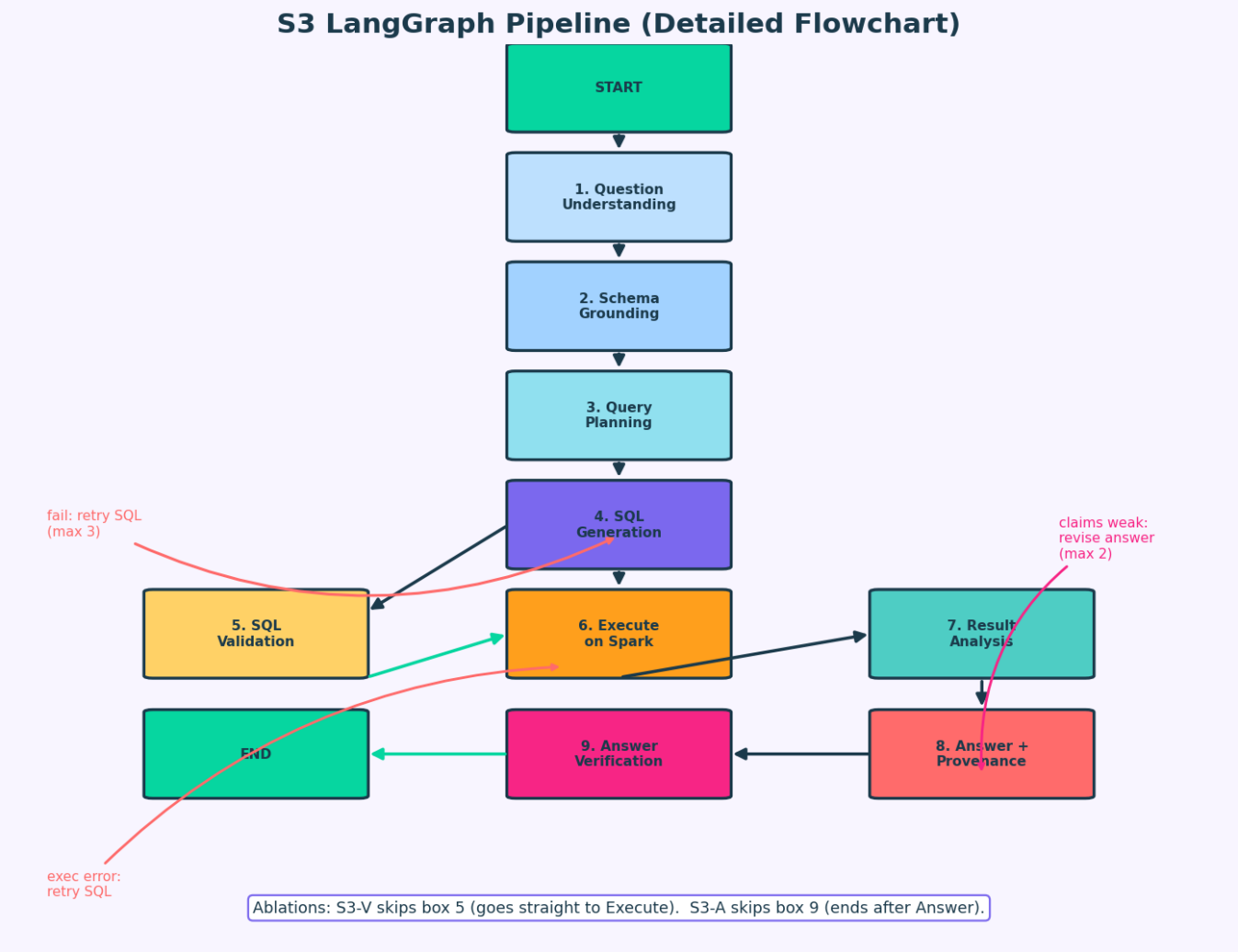


Shared pieces

* generate_sql() builds one SELECT using the schema text and optional error list.
* validation_for() / sqlglot rejects unsafe or unknown tables/columns.
* run_sql() executes with a row cap (default 200) and timeout.
* narrate() turns result rows into a readable answer.

7. S3 LangGraph flowchart (detailed)

LangGraph is a library for building AI workflows as a graph: boxes (nodes) and arrows (edges). Conditional arrows mean 'if this, go there'. S3 stores working memory in state (question, SQL, feedback, timings, token usage).



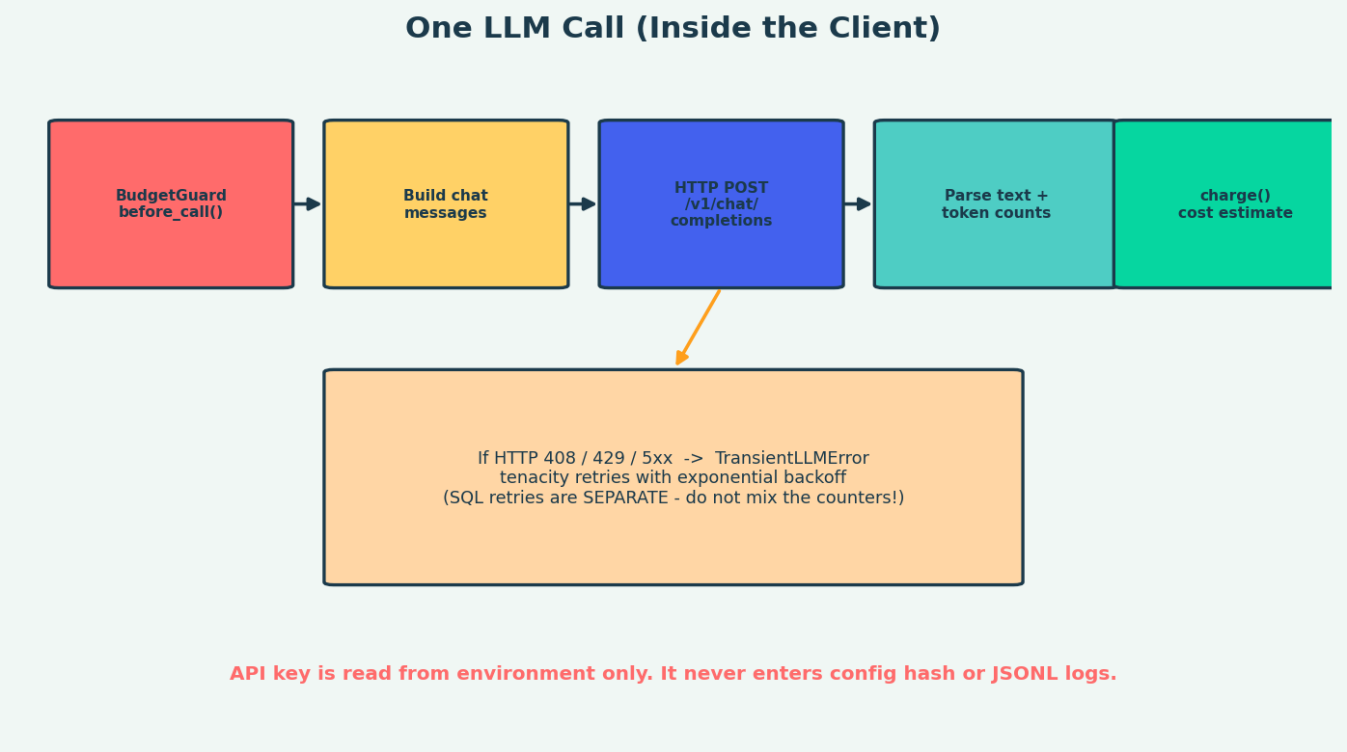
Ablation variants (science experiments)

* S3-V: turn OFF SQL validation - first SQL goes to execute. Shows value of the validator.
* S3-A: turn OFF answer verification - skip claim checking. Shows value of the final check.
* Flags change edges; node code stays the same. That is clean experiment design.

Student tip: recursion_limit is a safety net so the graph cannot loop forever. Business limits (max_sql_retries, max_answer_revisions) are the real policy knobs.

8. LLM client, prompts, and budget safety

LLM means Large Language Model (the AI text model). This project talks to an OpenAI-compatible chat API with the standard library (urllib), not a heavy SDK.



Prompts

Prompt files live in prompts/ (question understanding, schema grounding, SQL generation, narration, claim extraction, ...). Placeholders look like <<SCHEMA>> so JSON braces inside prompts stay safe. prompts/VERSION is stored on every run for reproducibility.

BudgetGuard

If you set a dollar budget, the guard estimates cost from token counts and pricing rates. It blocks the NEXT call when the cap is already reached. One call that started under the cap can still finish a little over, because tokens are known only after the response.

9. SQL validation (the safety gate)

Before trusting model SQL, a deterministic checker (sqlglot, Spark dialect) inspects it. No AI is involved in this check - that keeps messages reliable for retries.

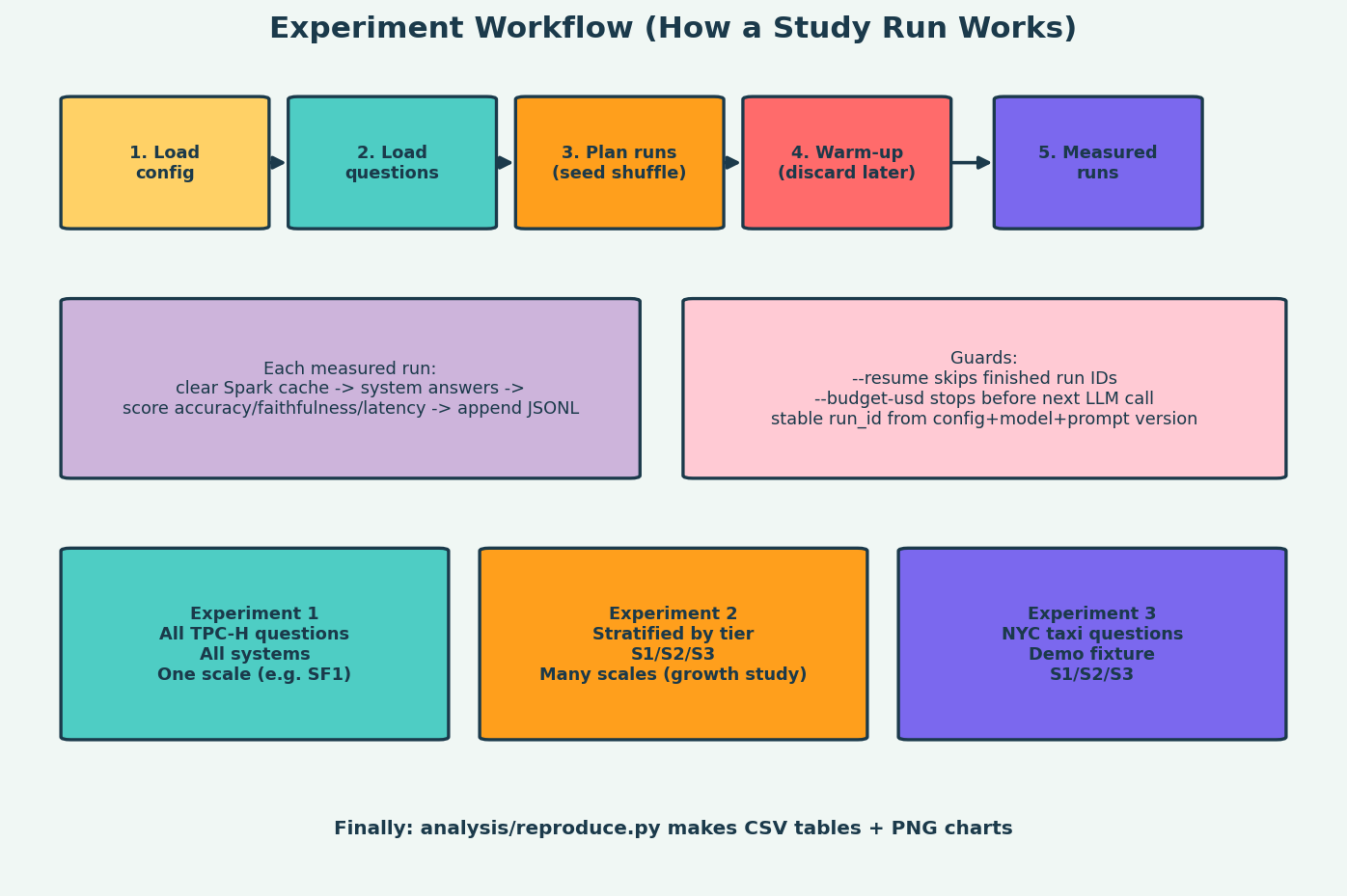
What gets rejected?

* More than one statement
* Anything that is not a read-only SELECT-family query (no DROP/UPDATE)
* Unknown tables or columns outside the business schema
* Ambiguous columns
* Cross joins / joins without a condition

Student tip: Passing validation does NOT mean the SQL is semantically correct. It can still filter the wrong year or join the wrong way. That is why we also score against gold answers.

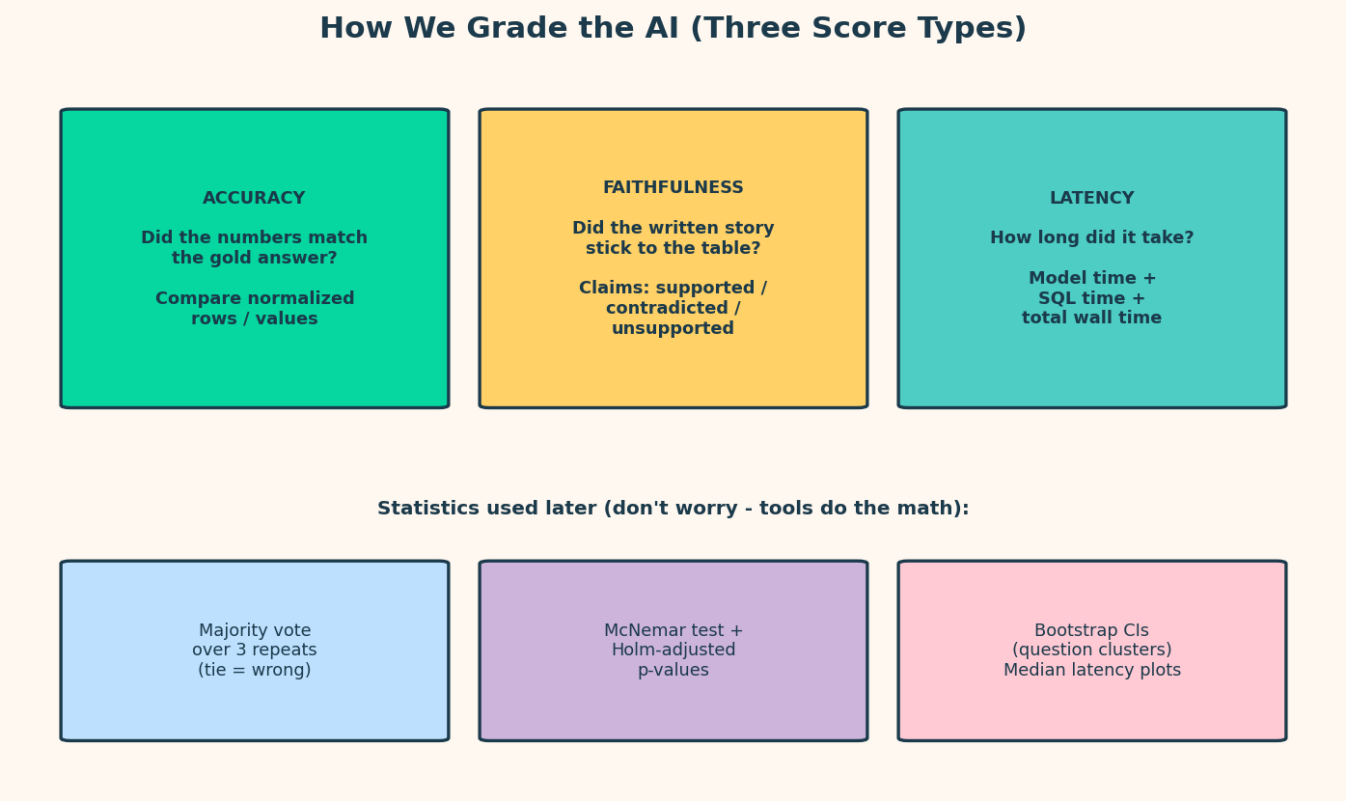
10. Experiment workflow

An experiment is a planned list of (question, system, scale, repeat) jobs. Order is shuffled with a seed so no system always runs first. One warm-up is recorded then dropped in analysis so cold-start noise does not spoil charts.



11. Evaluation: how answers are graded

Grading has three lenses. Accuracy asks 'are the numbers right?'. Faithfulness asks 'does the story stick to the table?'. Latency asks 'how long did we wait?'.



Faithfulness labels (easy meanings)

* supported - the claim matches the result evidence
* contradicted - the claim fights the evidence
* unsupported - claim has a checkable fact that is not in the result
* non_verifiable - soft explanation without a checkable number/comparison

12. Configuration and secrets

Where settings live

* configs/default.yaml - model name, temperature, paths, Spark caps, S3 flags, pricing rates
* .env - NLQ_LLM_API_KEY only (copy from .env.example)
* NLQ_* environment variables can override selected fields

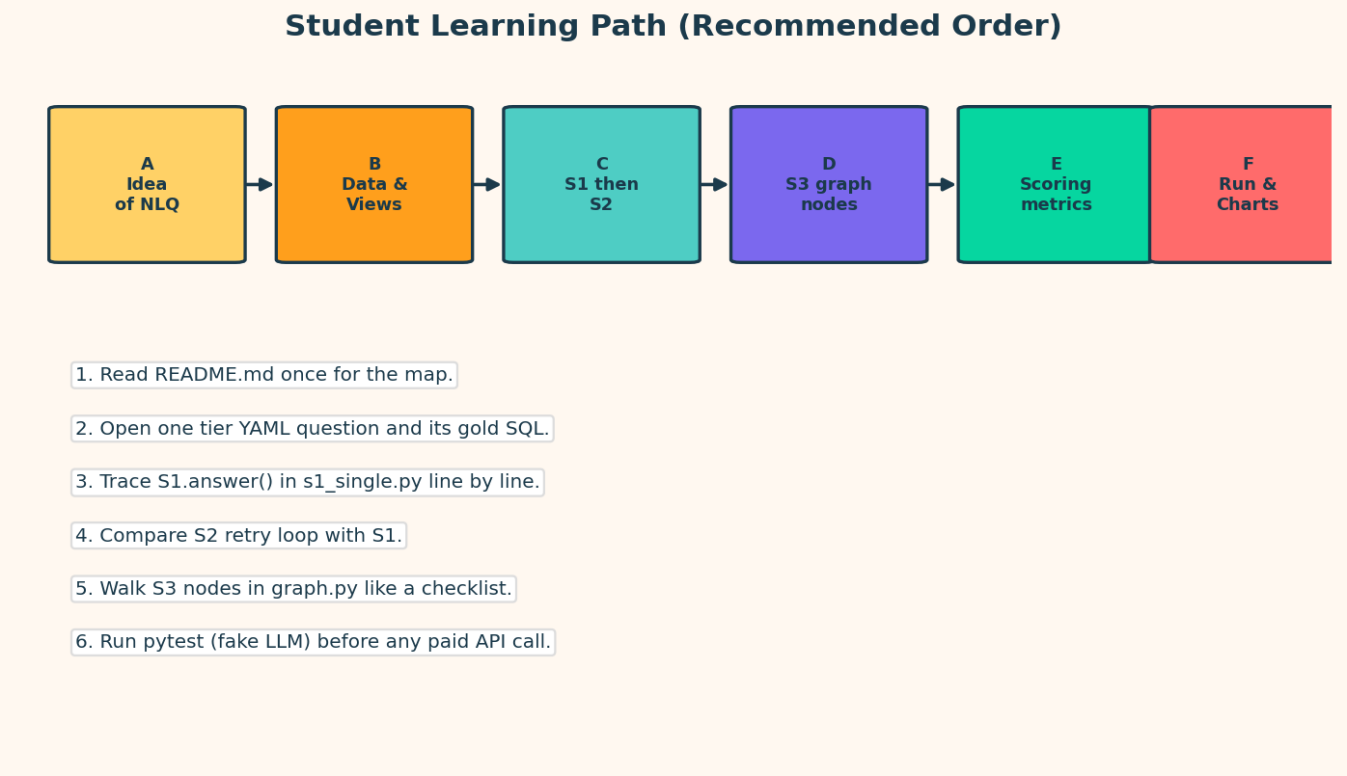
Config hash

AppConfig.config_hash() is a SHA-256 fingerprint of the resolved config. It helps group identical runs. Absolute paths mean the hash can change if you move the project folder. The API key is intentionally NOT part of the hashed config.

Student tip: Never commit a real API key. Tests use a scripted fake model, so CI can stay free.

13. Student learning path

Do not try to learn every file on day one. Follow this path. After step C you already understand most of the project idea.



Handy commands (after setup)

```
uv sync
uv run pytest
uv run python -m nlq.data.generate_tpch --scale 0.01
uv run python -m nlq.eval.run_experiment --experiment 1 --systems s1 s2 s3 --scale sf0.01 --repeats 1
uv run python -m nlq.analysis.reproduce --runs runs/experiment_1.jsonl --out results
```

14. Glossary (keep this open while reading code)

Agent / agentic: A workflow with multiple steps, memory/state, and decision loops - not one single prompt.

Ablation: Turn one feature off to measure how much it mattered.

Benchmark: A fixed set of questions with known answers for fair comparison.

Config hash: Fingerprint of settings used for a run.

DuckDB: Fast local analytics database engine.

Faithfulness: Whether the written answer sticks to computed evidence.

Gold answer: Trusted correct result used for scoring.

JSONL: Text file with one JSON object per line (easy to append and resume).

LangGraph: Library to build AI pipelines as graphs of nodes and edges.

Latency: How long a run takes (wall-clock and parts of it).

LLM: Large Language Model - the AI that writes SQL/text.

NLQ: Natural-Language Querying - ask databases in English.

Parquet: Columnar data file format used for TPC-H tables here.

Prompt: Instructions/template sent to the LLM.

Provenance: Trace of where an answer came from (SQL + evidence).

Scale factor: Size knob for TPC-H data (bigger = more rows).

Schema: Map of tables and columns the AI may use.

Spark: Distributed SQL/analytics engine used to run answering queries.

sqlglot: SQL parser/toolkit used for validation.

Temperature: Sampling randomness for the LLM; 0 is more stable for evals.

Text-to-SQL: Task of turning English into SQL.

Warm-up run: First throwaway run to reduce cold-start noise.

You made it. Re-open the diagrams whenever a code file feels confusing.